

于将.NET全面的编程技术和领域知识应用到代码中,防止错误的出现。

(2) 自动测试代码构造与功能。TEST智能特性,能提取代码、审查代码,生成测试用例,自动完成单元测试。TEST产生的单元测试可以由用户自定义。

(3) 通过自动衰减测试自动地维持代码完整性。

5. IBM Rational PurifyPlus

IBM Ration PurifyPlus 包含以下三种工具。

(1) PureCoverage 提供代码覆盖率分析,找出未经测试的程序代码,度量在所有测试用例中多少代码运行了,多少代码没有运行。

(2) Quantify 用来进行性能分析,识别应用程序性能瓶颈。

(3) Purify 用来进行内存分析,寻找应用程序的内存泄漏和错误的内存使用,这些有可能导致应用程序崩溃。

6. JProbe Suite

JProbe Suite 包括4个独立工具:Memory Debugger 和 Profiler 被称作程序性能工具,而 Threadalyzer 和 Coverage 被称作程序校正工具。

(1) Memory Debugger 可发现因不足参数管理和对象的过度分配而引起的内存泄漏。

(2) Profiler 通过统计程序各部分的运行时间帮助找出程序的性能瓶颈。

(3) Threadalyzer 通过找出程序中的死锁和空值以检验线程的正确性。

(4) Coverage 用来报告程序的测试覆盖率和未测试覆盖。

7. PRQA 单元测试工具

PRQA 提供的代码测试工具有 QA·C,QA·C++ 和 QA·J (<http://www.pr-qa.com/programmingresearch/PRODUCTS.html>),包括进行编码规则检查和静态分析,找出过于复杂、不便于移植和维护等各种代码质量问题。PRQA 公司提供编码规则检查工具包,包括 HIGH·INTEGRITY C++,QA·MISRA 和 QA·JSF++。PRQA 的静态测试工具可与 Vector 公司的动态测试工具 VectorCAST 很好地集成。而且,它还能够和 Headway 的 Structure101 很好地集成,使后者具有复杂 C/C++代码的结构分析以及质量度量能力。

5.7 系统集成的模式与方法

在软件开发中,经常会遇到这样的情况,单元测试时能确认每个模块都能单独工作,但这些模块集成在一起之后会出现有些模块不能正常工作的问题。这主要是因为模块相互调用时接口会引入新的问题,包括接口参数不匹配、传递错误数据、全局数据结构出现错误等。这时,需要进行集成测试(Integration Test)。集成测试是将已分别通过测试的单元按设计要求集成起来再进行的测试,以检查这些单元之间的接口是否存在问题,包括接口参数的一致性引用、业务流程端到端的正确性等。

集成测试既要求参与的人熟悉单元的内部细节,又要求能够从足够高的层次上观察整个系统。一般由有经验的测试人员和软件开发者共同完成集成测试的计划和执行。集成测试是白盒测试和黑盒测试相结合的典型应用场景。在自底向上集成的早期,白盒测试占较大的比例,随着集成测试的规模越来越大,白盒测试所占的比重在逐步减少,渐渐地黑盒测试占据主导地位。

5.7.1 集成测试的模式

在开始集成测试时,首先需要选择集成模式。集成模式是软件集成测试中的策略体现,其重要性是明显的,直接关系到测试的效率、结果等,一般要根据具体的系统来决定采用哪种模式。集成测试基本可以概括为以下两种。

(1) 非渐增式测试模式:先分别测试每个模块,再把所有模块按设计要求放在一起结合成所要的程序,如大棒模式。

(2) 渐增式测试模式:把下一个要测试的模块同已经测试好的模块结合起来进行测试,测试是在模块一个一个的扩展下进行,其测试的范围逐步增大。

在非增量式集成中容易出现混乱,因为测试时可能发现一大堆错误,为每个错误定位和纠正非常困难,并且在改正一个错误的同时又可能引入新的错误,新旧错误混杂,更难断定出错的原因和位置。与之相反的是增量式集成模式,程序一段一段地扩展,每次测试的接口非常有限,错误易于定位和纠正,界面的测试也可做到完全彻底。在两种模式中,增量式集成模式有一定的优势,但它们有各自的优缺点。

(1) 渐增式测试模式需要编写的软件较多,工作量较大,而非渐增式测试开销小。

(2) 渐增式测试模式发现模块间接口错误早;而非渐增式测试模式晚。

(3) 非渐增式测试模式发现错误,较难诊断;而使用渐增式测试模式,如果发生错误则往往和最近加进来的那个模块有关。

(4) 渐增式测试模式测试更彻底。

(5) 渐增式测试模式需要较多的机器时间。

(6) 使用非渐增式测试模式,可以并行测试。

在实际工作中,一般采用渐增式测试模式,具体的实践有自顶向下、自底向上、混合策略等。

5.7.2 自顶向下和自底向上集成方法

1. 自顶向下法

自顶向下法(Top-down Integration),从主控模块(“主程序”)开始,沿着软件的控制层次向下移动,从而逐渐把各个模块结合起来。在集成过程中,可以使用深度优先的策略或宽度优先的策略,如图 5-13 所示,其具体步骤如下。

(1) 对主控模块进行测试,测试时用桩程序代替所有直接隶属于主控模块的模块。

(2) 根据选定的结合策略(深度优先或宽度优先),每次用一个实际模块代替一个桩程序(新结合进来的模块往往又需要新的桩程序)。

(3) 在结合下一个模块的同时进行测试。

(4) 为了保证加入模块没有引进新的错误,可能需要进行回归测试(即全部或部分地重复以前做过的测试)。

从第(2)步开始不断地重复进行上述过程,直至完成。自顶向下法的主要优点是:不需要测试驱动程序,能够在测试阶段的早期实现并验证系统的主要功能,而且能在早期发现上层模块的接口错误。其缺点是:需要桩程序,可能遇到与此相联系的测试困难,低层关键模块中的

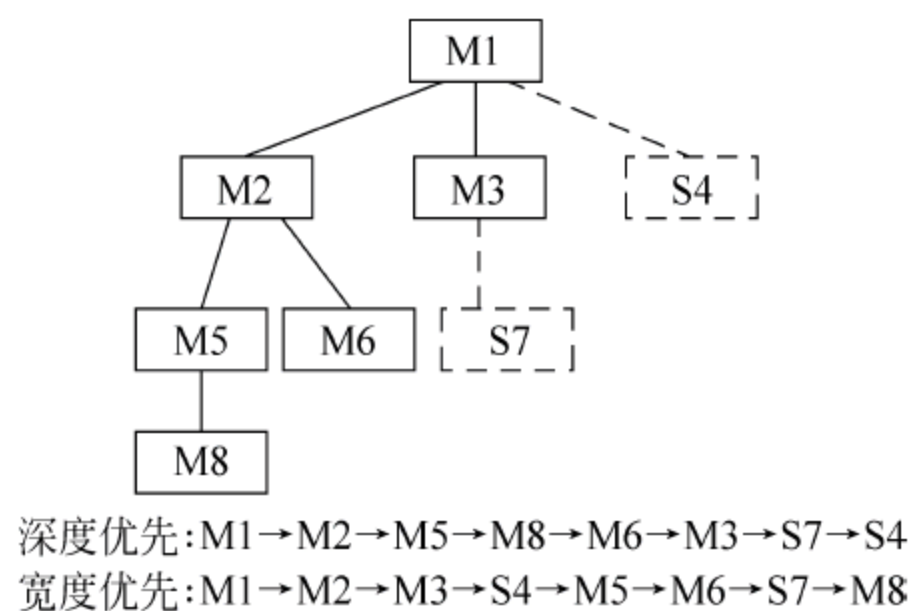


图 5-13 自顶向下集成方法示意图

错误发现较晚,而且用这种方法在早期不能充分展开人力。

2. 自底向上法

自底向上(Bottom-up Integration)测试从“原子”模块(即在软件结构最底层的模块)开始集成以进行测试,如图 5-14 所示,具体策略如下。

- (1) 把底层模块组合成实现某个特定的软件子功能的族。
- (2) 写一个驱动程序(用于测试的控制程序),协调测试数据的输入和输出。
- (3) 对由模块组成的子功能族进行测试。
- (4) 去掉驱动程序,沿软件结构自下向上移动,把子功能族组合起来形成更大的子功能族(Cluster)。

从第(2)步开始不断地重复进行上述过程,直至完成。

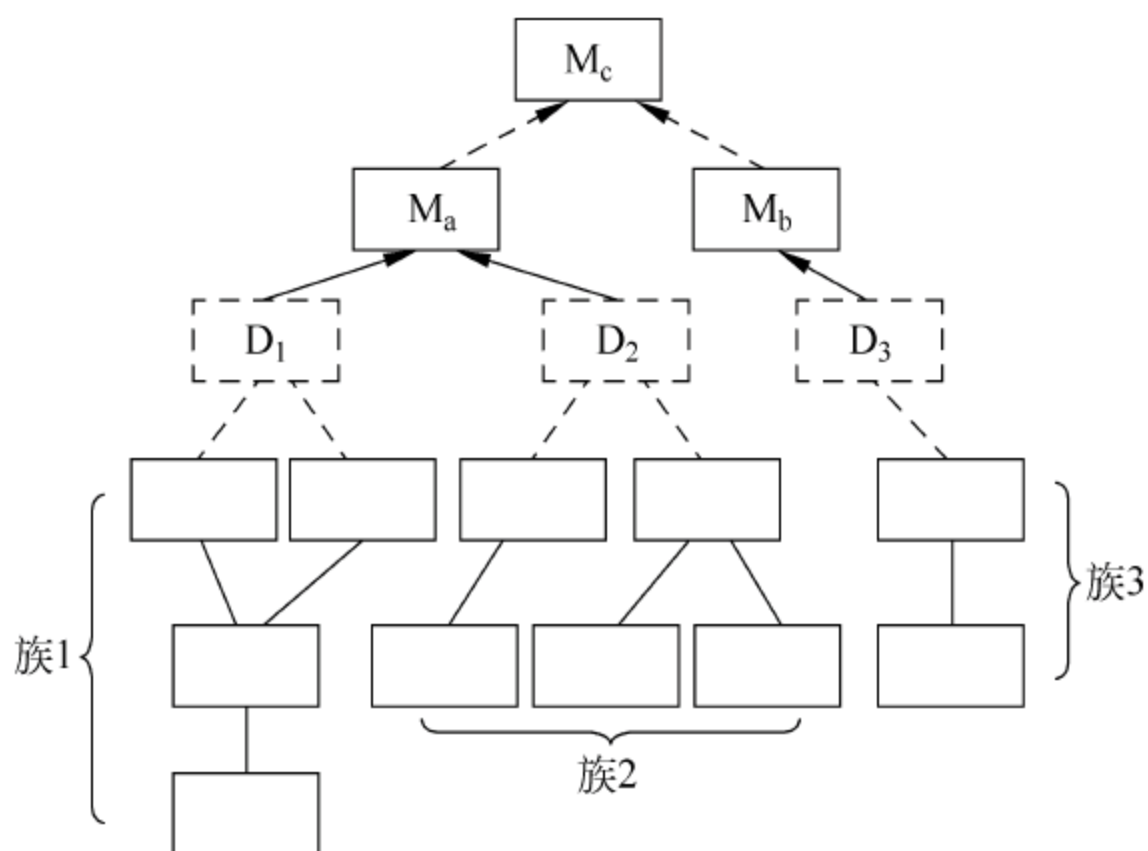


图 5-14 自底向上集成方法示意图

自底向上法的优缺点与自顶向下法刚好相反。

5.7.3 混合策略

在具体测试中,可采用混合策略,即结合上述的两种方法——自顶向下法和自底向上法来逐步实施集成测试。

- (1) 改进的自顶向下法: 基本使用“自顶向下”法,但在测试早期,使用“自底向上”法测试少数的关键模块。
- (2) 混合法: 对软件结构中较上层,使用的是“自顶向下”法;对软件结构中较下层,使用的是“自底向上”法,两者相结合,如图 5-15 所示。

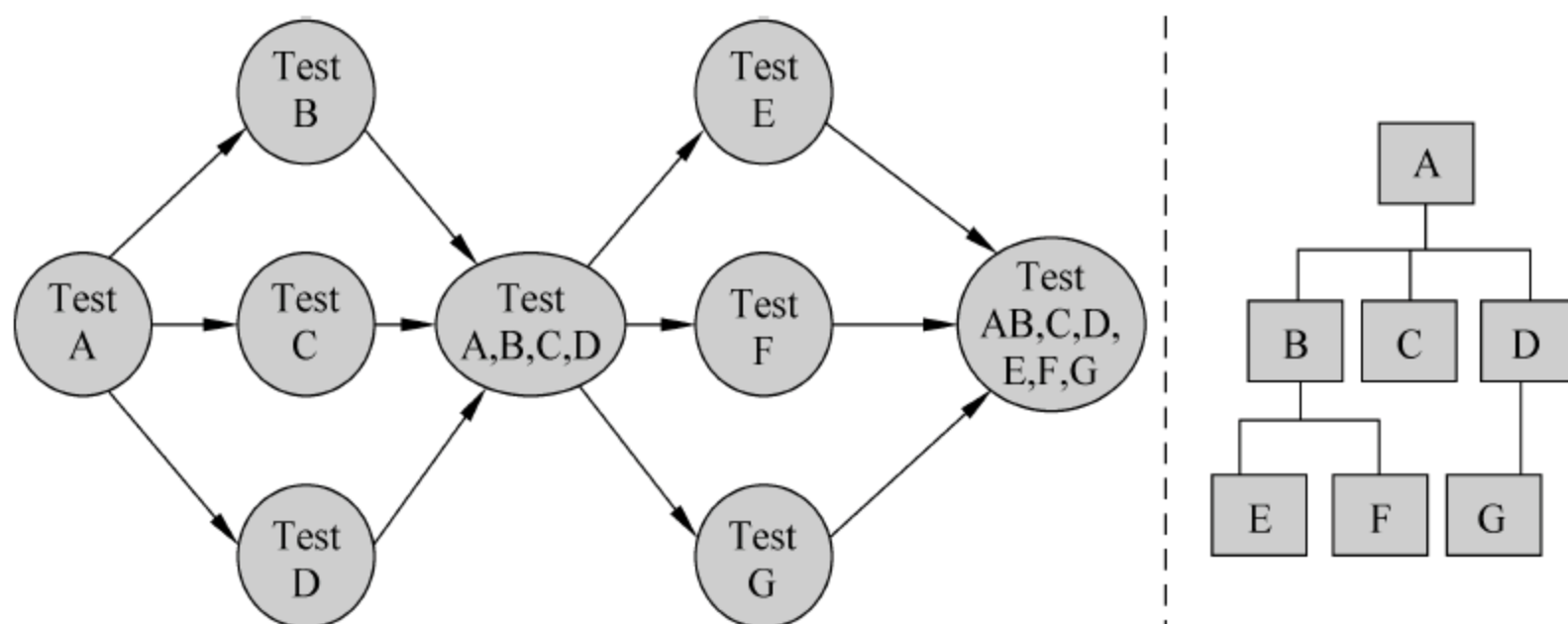


图 5-15 混合策略集成示意图

这种混合策略也有一些不同的组合方式,如三明治集成方法(Sandwich Integration),基本思想是一致的,自两头向中间集成,只是具体实现有些差异,如图 5-16 所示。

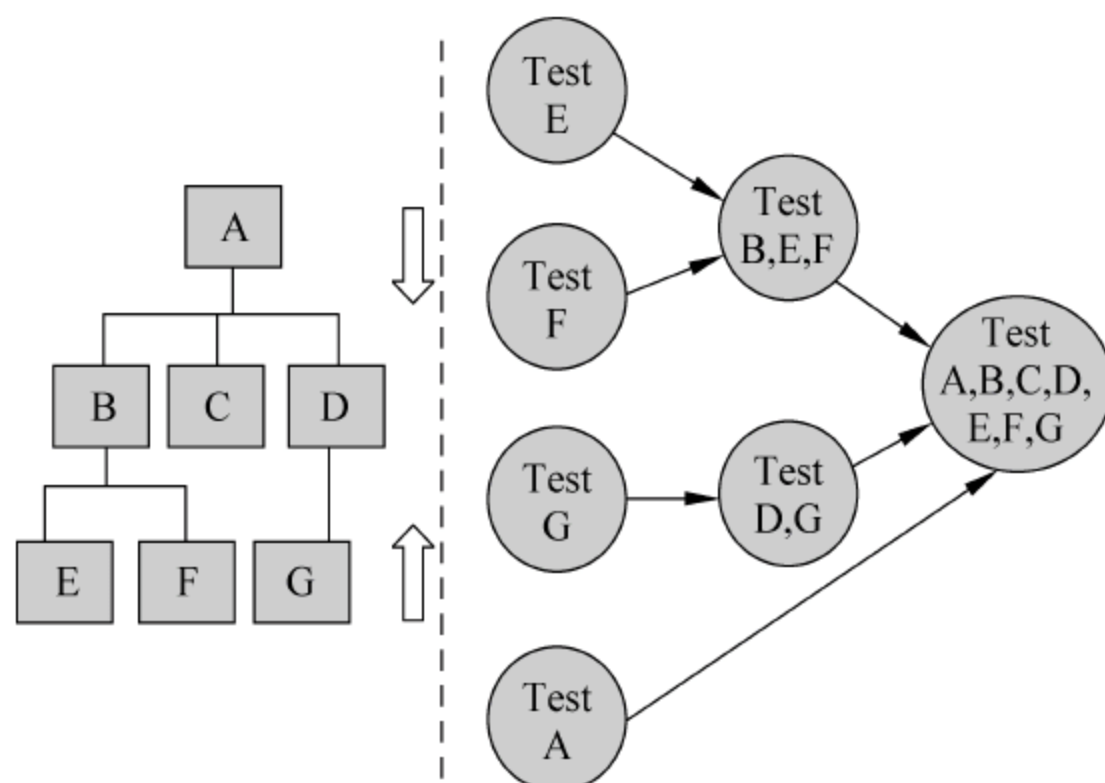


图 5-16 三明治集成方法示意图

采用三明治方法的优点是：它将自顶向下和自底向上的集成方法有机地结合起来,不需要写桩程序,因为在测试初自底向上集成已经验证了底层模块的正确性。采用这种方法的主要缺点是：在真正集成之前每一个独立的模块没有完全测试过。

5.7.4 持续集成

通常系统集成都会采用持续集成的策略,软件开发中各个模块不是同时完成,根据进度将完成的模块尽可能早地进行集成,有助于尽早发现 Bug,避免集成中大量 Bug 涌现。同时自底向上集成时,先期完成的模块将是后期模块的桩程序,而自顶向下集成时,先期完成的模块将是后期模块的驱动程序,从而使后期模块的单元测试和集成测试出现了部分的交叉,不仅节省了测试代码的编写,也有利于提高工作效率。

在没有采用持续集成策略的开发中,开发人员经常需要集中开会来分析软件究竟在什么地方出了错。因为某个程序员在写自己这个模块代码时,可能会影响其他模块的代码,造成与已有程序的变量冲突、接口错误,结果导致被影响的人还不知道发生了什么,Bug 就出现了。这种 Bug 是最难查的,因为问题不是出在某一个人的领域里,而是出在两个人的交流上面。随着时间的推移,问题会逐渐恶化。通常,在集成阶段出现的 Bug 早在几周甚至几个月之前就已经存在了。结果,开发者需要在集成阶段耗费大量的时间和精力来寻找这些 Bug 的根源。

如果使用持续集成,这样的 Bug 绝大多数都可以在引入的第一天就被发现。而且,由于一天之中发生变动的部分并不多,所以可以很快找到出错的位置。如果找不到 Bug 究竟在哪里,也可以不把这些代码集成到产品中去。所以,持续集成可以减少集成阶段消灭 Bug 所消耗的时间,从而最终提高软件开发的质量与效率。

为了做到持续集成测试,需要构建良好的集成测试环境。例如,业界通常采用 Maven、Ant 在 Jenkins 中完成持续构建和集成,然后在此基础上自动触发自动化测试,完成基本的功能和接口测试,一般称为构建包的验证(Build Verification Test,BVT)。这种 BVT 可以看作是非严格意义上的集成测试,因为如果集成有问题,会在版本构建中出现问题,会被 BVT 发现。基于良好的基础设施,就可以做到持续构建、持续集成测试。

小结

单元测试的对象是在程序系统中的最小单元——模块或组件上,其目标不仅是测试代码的功能性,还需确保代码在结构上可靠且健全。单元测试也可以采用静态测试和动态测试。静态测试主要体现在两个方面,一方面通过工具进行自动分析,另一方面可以通过人工评审,最常用的方法是互为评审(Peer Review),对一些关键代码或新人写的代码,主要采用走查(Walk Through)和会议审查(Inspection)等评审方式。另一方面,可以借助静态测试工具来完成对所有代码的扫描和分析,输出测试报告,这种方式的应用越来越多。

动态技术主要采用基于代码的逻辑覆盖方法,从程序的内部结构出发设计测试用例,检查程序模块或组件已实现的功能与定义的功能是否一致,并结合基于输入域的测试方法、组合测试方法等,完成对调用参数、变量取值等测试,最终完成控制流和数据流的分析与测试。由于模块规模小、功能单一、逻辑简单,测试人员有可能通过模块说明书和源程序,清楚地了解该模块的 I/O 条件和模块的逻辑结构,采用结构测试(白盒法)的用例,尽可能达到彻底测试,使之对任何合理和不合理的输入都能鉴别和响应。

本章还介绍了单元测试中常用的测试工具,包括开源工具和商业工具。重点介绍了单元测试框架 JUnit、代码静态分析工具 CheckStyle/PMD 与 FindBug、代码复杂度检测工具 SourceMonitor,已经介绍如何用 JUnit+Ant 构建自动测试等。通过使用这些不同方面的单元测试工具,不仅更容易实施单元测试,不断复用测试脚本,减少单元测试的工作量,而且借助静态分析、复杂度衡量等,可以更好地清楚缺陷,提供代码的质量。

单元测试和集成测试紧密相关,几乎同步进行,而且目前业界都提倡持续集成、持续测试。在持续测试中,更关注集成测试的基础设施,从而能够比较容易地完成持续构建和持续集成测试。

思考题

1. 为什么要进行单元测试? 单元测试的主要任务有哪些?
2. 单元测试的对象不可能是一组函数或多个程序的组合,为什么?
3. 单元测试一般由开发人员完成,并采用白盒测试技术,这样会获得更高的测试效率和更彻底的测试,谈谈其中的道理。
4. 代码评审有哪些方法? 哪一种方法比较有效? 为什么?
5. 如何做好单元测试的各个阶段的管理工作?
6. 动手写一个类的 JUnit 单元测试方法,并让其运行成功,体验 JUnit 的使用方法。
7. CheckStyle/PMD 与 FindBugs 各自的主要功能是什么? 试着在某个 JavaEE 项目中使用 FindBugs 进行检测,并分析测试结果。
8. SourceMonitor 的主要作用是什么? 试着对一个已有的项目用 SourceMonitor 进行分析。
9. 进一步了解持续集成和持续测试的知识,设法自己搭建这样的集成环境。